Promemoria: hook di Claude Code e MCP

Da tenere accanto durante l'attività. Letto sulla documentazione ufficiale il 9/11/2026; nomi e dettagli cambiano in fretta, ricontrollali. Fonti: https://code.claude.com/docs/en/hooks , https://code.claude.com/docs/en/hooks-guide , https://code.claude.com/docs/en/mcp , https://modelcontextprotocol.io

Gli eventi più usati

Evento	Quando scatta	Può bloccare?
`PreToolUse`	prima che l'agente usi uno strumento	sì (uscita 2)
`PostToolUse`	dopo che lo strumento ha girato	no: l'azione è già avvenuta, l'agente legge il messaggio
`UserPromptSubmit`	quando invii un messaggio	sì
`Stop`	quando l'agente finisce di rispondere	sì: lo fa continuare
`SessionStart`	all'avvio o alla ripresa della sessione	no: aggiunge solo contesto
`Notification`	quando l'agente aspetta te	no

Gli eventi sono molti di più: l'elenco completo è nel riferimento.

Cosa riceve lo hook (standard input, JSON)

```
{
  "hook_event_name": "PreToolUse",
  "session_id": "abc123",
  "cwd": "C:\\palestra",
  "tool_name": "Bash",
  "tool_input": { "command": "git status" }
}
```

L'evento vero ha altri campi. Per Bash c'è `tool_input.command`; per Edit e Write c'è `tool_input.file_path`.

Come risponde lo hook (PreToolUse)

Uscita	Effetto
`0`	nessuna obiezione: il comando segue il normale percorso dei permessi
`2`	bloccato; quello che scrivi sullo standard error arriva all'agente
qualsiasi altro, anche `1`	errore che **non** blocca: il comando parte

Il motivo del blocco va sullo **standard error** (`console.error`), non sullo standard output.

Configurazione (`.claude/settings.json`)

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          { "type": "command",
            "command": "node \"$CLAUDE_PROJECT_DIR/.claude/hooks/blocca-
comandi.js\"" }
        ]
      }
    ]
  }
}
```

- Tre livelli: evento, gruppo con `matcher`, gestori.
- `matcher` distingue maiuscole e minuscole: `Bash`, `Edit|Write`, `mcp__server__.*`.
- `$CLAUDE_PROJECT_DIR` è la radice del progetto.
- Il JSON non ammette virgole finali né commenti.

Provare uno hook

A mano, senza agente:

```
echo '{"tool_name":"Bash","tool_input":{"command":"ls"}}' | node
.claude/hooks/blocca-comandi.js
echo $?
```

In PowerShell il codice di uscita si legge con `$LASTEXITCODE`. Dentro Claude Code: `/hooks` mostra gli hook attivi, `Ctrl+O` la trascrizione, `claude --debug` i dettagli.

MCP in breve

- **Host** (Claude Code) → un **client** per ogni **server** → il sistema esterno.
- Un server offre **strumenti**, **risorse**, **prompt**. Si collega con **stdio** (programma locale) o **HTTP** (servizio remoto).
- Ambiti in Claude Code: `local` (solo tu, questo progetto), `project` (file `.mcp.json`, condiviso con git), `user` (tutti i tuoi progetti).

Server di file in sola lettura su una cartella di prova (richiede Node.js 24 o più recente):

```
claude mcp add --transport stdio documenti -- npx -y @modelcontextprotocol/server-
filesystem --read-only ./cartella-prova
```

Comandi utili: `claude mcp list`, `claude mcp get documenti`, `claude mcp remove documenti`; dentro la sessione `/mcp`. I permessi di uno strumento MCP si scrivono con il nome `mcp__server__strumento`.

Prima di collegare un server

- Lo conosci e ti fidi di chi lo ha scritto?
- Che cosa può leggere? Che cosa può scrivere? Basta la sola lettura?
- Il testo che restituisce può venire da altri (pagine, segnalazioni)? Allora è un input non fidato.
- Token e chiavi stanno nelle variabili d'ambiente, mai dentro `.mcp.json`.
